



VISUAL ANALYTICS: NOVEL VISUALIZATIONS WITH D3.JS

AB MOSCA (THEY/THEM)



TODAY'S PLAN

- Overview
- SVG
- D3 Basics

D3.JS OVERVIEW

- D3 → Data Driven Documents; it is a JavaScript library for creating custom, novel, interactive visualizations on the web
- Good resource: <https://www.d3indepth.com/introduction/>
- D3 includes built in functions to bind data with HTML SVG elements

SVG

SVG stands for Scalable Vector Graphics.

SVG defines vector-based graphics in XML format.

SVG graphics are scalable, and do not lose any quality if they are zoomed or resized.

SVG is supported by all major browsers.

SVG

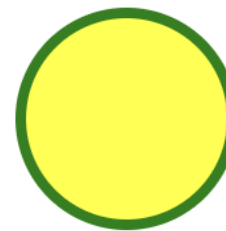
```
<!DOCTYPE html>
<html>
<body>

<h1>My first SVG</h1>

<svg width="100" height="100"
xmlns="http://www.w3.org/2000/svg">
  <circle cx="50" cy="50" r="40" stroke="green" stroke-
width="4" fill="yellow" />
  Sorry, your browser does not support inline SVG.
</svg>

</body>
</html>
```

My first SVG



SVG

SVG Circle - <circle>

The `<circle>` element is used to create a circle.

The `<circle>` element has three basic attributes to position and set the size.

Attribute	Description
<code>r</code>	Required. The radius of the circle
<code>cx</code>	The x-axis center of the circle. Default is 0
<code>cy</code>	The y-axis center of the circle. Default is 0

```
<!DOCTYPE html>
<html>
<body>

<h1>My first SVG</h1>

<svg width="100" height="100"
xmlns="http://www.w3.org/2000/svg">
  <circle cx="50" cy="50" r="40" stroke="green" stroke-
width="4" fill="yellow" />
  Sorry, your browser does not support inline SVG.
</svg>

</body>
</html>
```

SVG

SVG Rectangle - <rect>

The `<rect>` element is used to create a rectangle and variations of a rectangle shape.

The `<rect>` element has six basic attributes to position and shape the rectangle:

Attribute	Description
width	Required. The width of the rectangle
height	Required. The height of the rectangle
x	The x-position for the top-left corner of the rectangle
y	The y-position for the top-left corner of the rectangle
rx	The x radius of the corners of the rectangle (used to round the corners). Default is 0
ry	The y radius of the corners of the rectangle (used to round the corners). Default is 0

SVG

SVG Rectangle - <rect>

The `<rect>` element is used to create a rectangle and

The `<rect>` element has six basic attributes to position

Attribute	Description
width	Required. The width of the rectangle
height	Required. The height of the rectangle
x	The x-position for the top-left corner of the rectangle
y	The y-position for the top-left corner of the rectangle
rx	The x radius of the corners of the rectangle (used to round the corners). Default is 0
ry	The y radius of the corners of the rectangle (used to round the corners). Default is 0

```
<!DOCTYPE html>
<html>
<body>

<h2>SVG rect Element</h2>

<svg width="300" height="130"
xmlns="http://www.w3.org/2000/svg">
  <rect width="200" height="100" x="10" y="10" rx="20"
ry="20" fill="blue" />
  Sorry, your browser does not support inline SVG.
</svg>

</body>
</html>
```

SVG rect Element



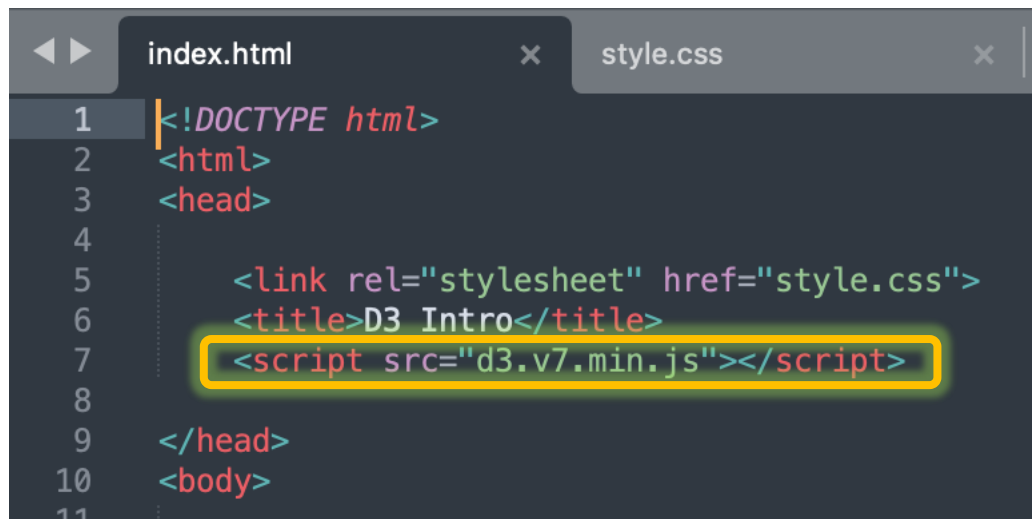
SVG Shapes

SVG has some predefined shape elements that can be used by developers:

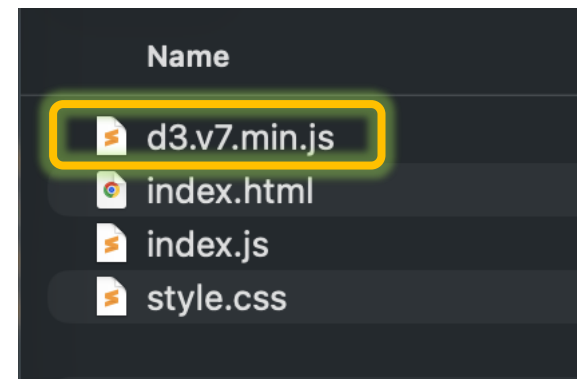
- Rectangle `<rect>`
- Circle `<circle>`
- Ellipse `<ellipse>`
- Line `<line>`
- Polyline `<polyline>`
- Polygon `<polygon>`
- Path `<path>`

D3 BASICS

- Starter code on the website
- Final code also on the website
- d3.js is a code library
- To use the library, include d3.js in your html with the `<script>` tags



```
1 <!DOCTYPE html>
2 <html>
3 <head>
4
5     <link rel="stylesheet" href="style.css">
6     <title>D3 Intro</title>
7     <script src="d3.v7.min.js"></script>
8
9 </head>
10 <body>
11
```



D3 BASICS: SELECTIONS

- Use `selectAll()` to select a group of SVGs
- Specify what to select via CSS selectors – Ex. `selectAll('circle')`, or `selectAll(".myCircle")`
- Use `select()` to select just the first matching element

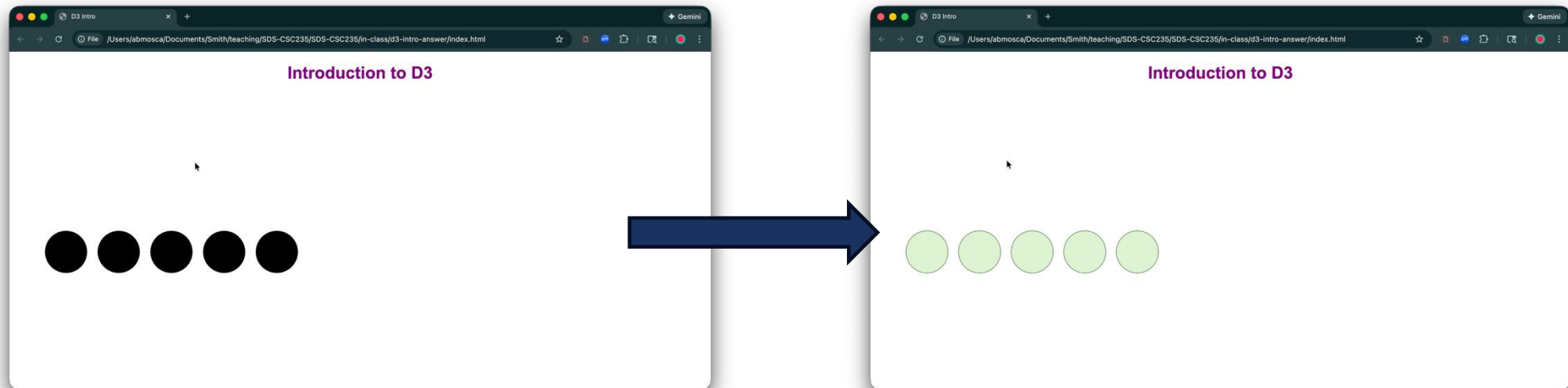
D3 BASICS: SELECTIONS

- After selecting elements, use D3 methods to modify them

Name	Behaviour	Example
<code>.style</code>	Update the style	<code>selectAll('circle').style('fill', 'red')</code>
<code>.attr</code>	Update an attribute	<code>selectAll('rect').attr('width', 10)</code>
<code>.classed</code>	Add/remove a class attribute	<code>select('.item').classed('selected', true)</code>
<code>.property</code>	Update an element's property	<code>selectAll('.checkbox').property('checked', false)</code>
<code>.text</code>	Update the text content	<code>select('div.title').text('My new book')</code>
<code>.html</code>	Change the html content	<code>select('.legend').html('<div class="block"></div><div>0 - 10</div>')</code>

D3 BASICS: SELECTIONS

- Your turn:
 - Use D3 selections to select all circles in the starter code and assign them the class 'selected'
 - Use CSS to style selected elements such that they are light green with a dark green border



D3 BASICS: DATA JOINS

- A data join creates a correspondence between an array of data and a selection of HTML or SVG elements.
- Joining an array to HTML/SVG elements means that:
 - HTML (or SVG) elements are added or removed such that **each array element has a corresponding HTML (or SVG) element**
 - each HTML/SVG element may be positioned, sized and styled according to the value of its corresponding array element

D3 BASICS: DATA JOINS

- Typically, four methods are used in a data join:
 - `.select` defines the element that'll act as a container (or parent) to the joined HTML/SVG elements
 - `.selectAll` defines the type of element that'll be joined to each array element
 - `.data` defines the array that's being joined
 - `.join` performs the join. This is where HTML or SVG elements are added and removed

```
28     <div>
29         <svg width="600" height="100">
30             <g id="container"></g>
31         </svg>
32     </div>
```

```
9
10 let arr = [1, 2, 3, 4, 5];
11
12 d3.select("#container")
13   .selectAll("rect")
14   .data(arr)
15   .join("rect");
```

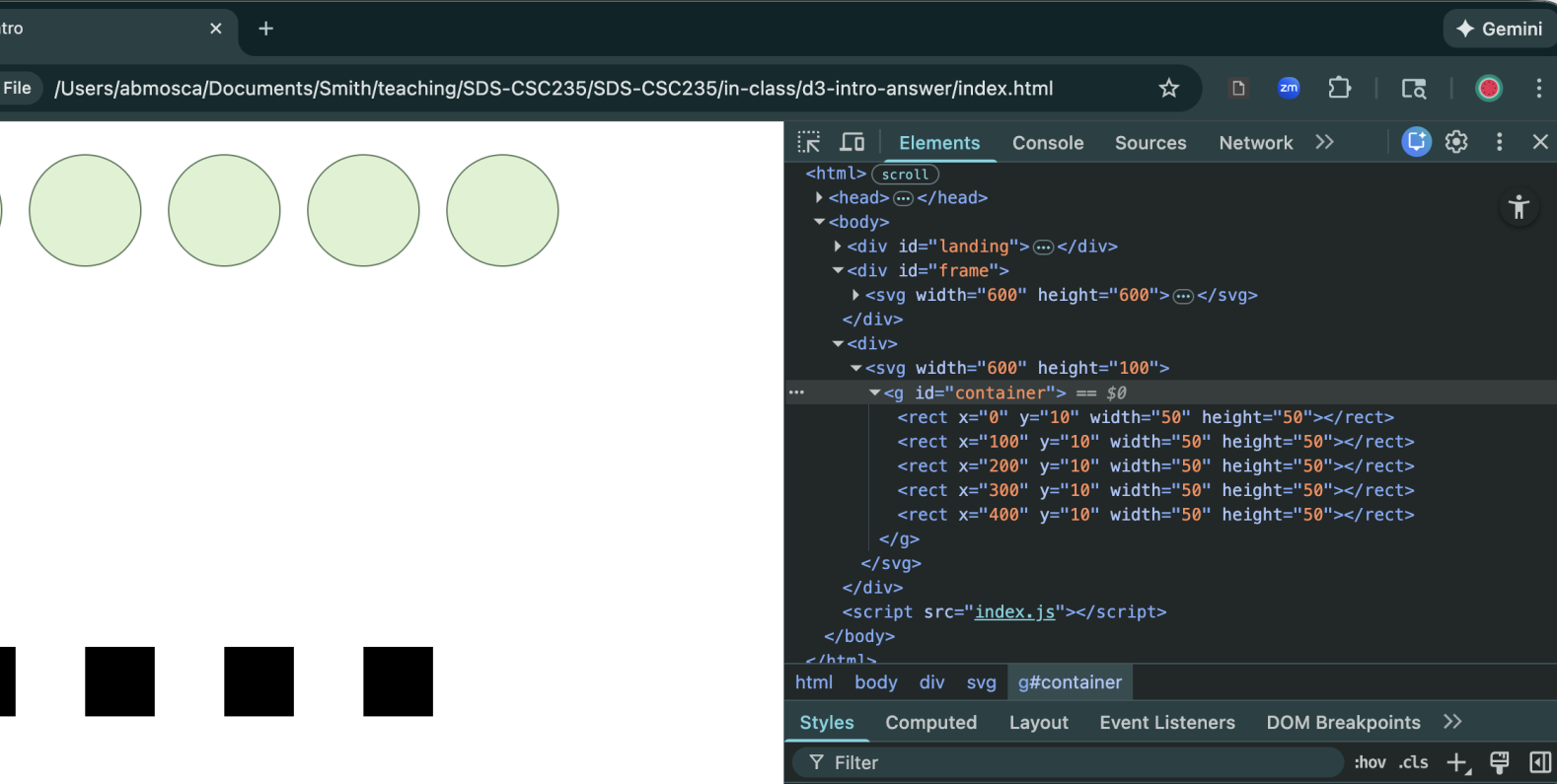
```
<!DOCTYPE html>
<html> scroll
  <head> ... </head>
  <body>
    <div id="landing"> ... </div>
    <div id="frame"> ... </div>
    <div>
      <svg width="600" height="100">
        ... <g id="container"> == $0
          <rect></rect>
          <rect></rect>
          <rect></rect>
          <rect></rect>
          <rect></rect>
        </g>
      </svg>
```

<https://www.d3indepth.com/introduction/>

D3 BASICS: DATA JOINS

- Once data and HTML/SVG elements are joined, we can use the data to dynamically set properties of the elements

```
9
10 let arr = [1, 2, 3, 4, 5];
11
12 d3.select("#container")
13   .selectAll("rect")
14   .data(arr)
15   .join("rect")
16   .attr("x", function(d, i) {
17     return i*100;
18   })
19   .attr("y", 10)
20   .attr("width", 50)
21   .attr("height", 50);
```



The browser window shows a visualization of five light green circles arranged horizontally. Below them are five black squares. The browser's developer tools are open, showing the HTML structure with a `<div id="container">` containing a `<svg>` with a `<g id="container">` containing five `<rect>` elements. The console and styles panels are also visible.

D3 BASICS: DATA JOINS

- Data will typically be in JSON format

```
22
23   let myData = [
24       {name: "apple", price: 1.50},
25       {name: "orange", price: 1.90},
26       {name: "pear", price: 2.00}
27   ]
28
```

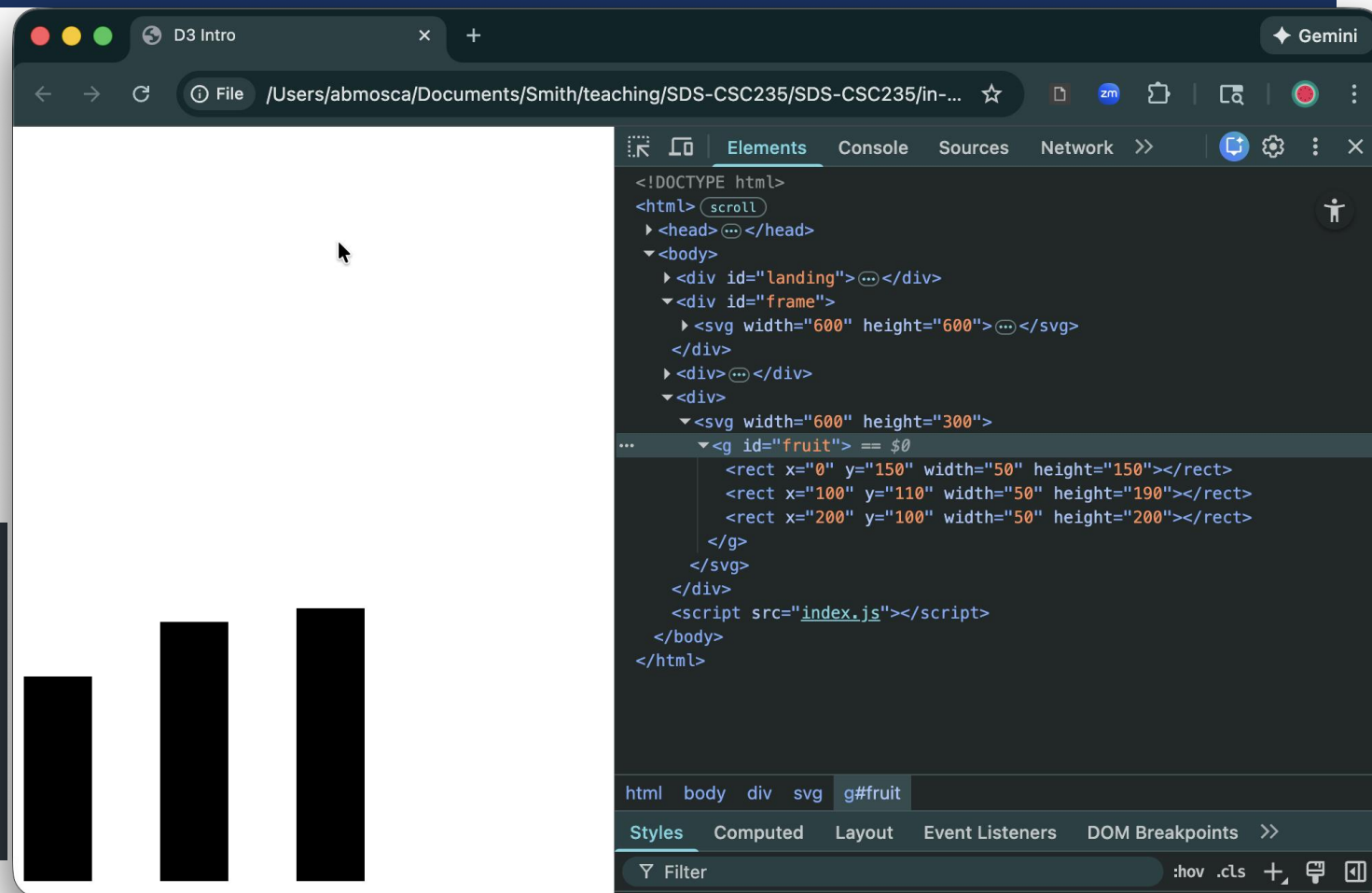
- In this format, each observation is an object. Data is accessed as properties of the object

```
38   .attr("height", function(d, i) {
39       console.log(d.name);
40       console.log(d.price);
41       return d.price * 100;
42   } );
43
```

D3 BASICS: DATA JOINS

- Your turn:
 - Add a rectangle for each observation in myData to your HTML
 - Use even horizontal spacing, and map height to price

```
let myData = [  
  {name: "apple", price: 1.50},  
  {name: "orange", price: 1.90},  
  {name: "pear", price: 2.00}  
]
```



D3 BASICS: AXES

- To make perceptually accurate visualizations, we need to map data values to visual channels such as position, color, length, etc.
- D3 has built in scale methods to make this easier
- Scale methods:
 - take an input (usually a number, date or category) and
 - return a value (such as a coordinate, a color, a length, or a radius)

■ Ex.

```
myScale(0);    // returns 0
myScale(2);    // returns 120
myScale(3);    // returns 180
...
myScale(10);   // returns 600
```

0 2 3 5 7.5 9 10

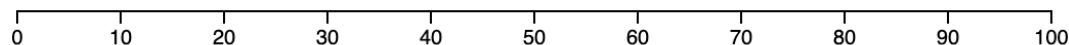
D3 BASICS: AXES

- To create an axis we need:
 - an SVG element to contain the axis (usually a g element)
 - a D3 scale function

```
<svg width="600" height="100">  
  <g transform="translate(20, 50)"></g>  
</svg>
```

```
let scale = scaleLinear().domain([0, 100]).range([0, 500]);  
  
let axis = axisBottom(scale);  
  
select('svg g')  
  .call(axis);
```

.call takes a function that operates on a selection as a parameter

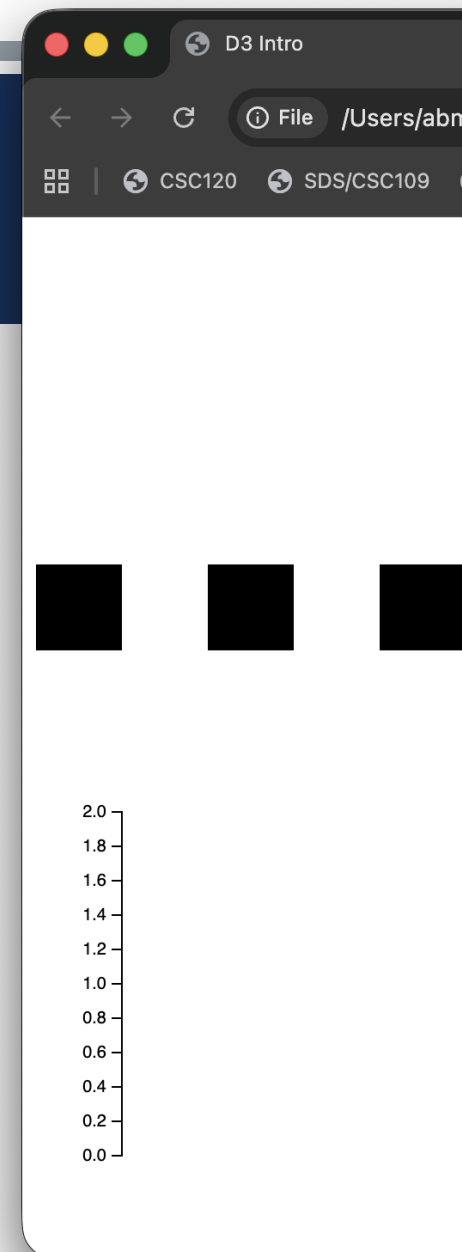


D3 BASICS: AXES

- Aside: we can dynamically add SVG containers to HTML using D3

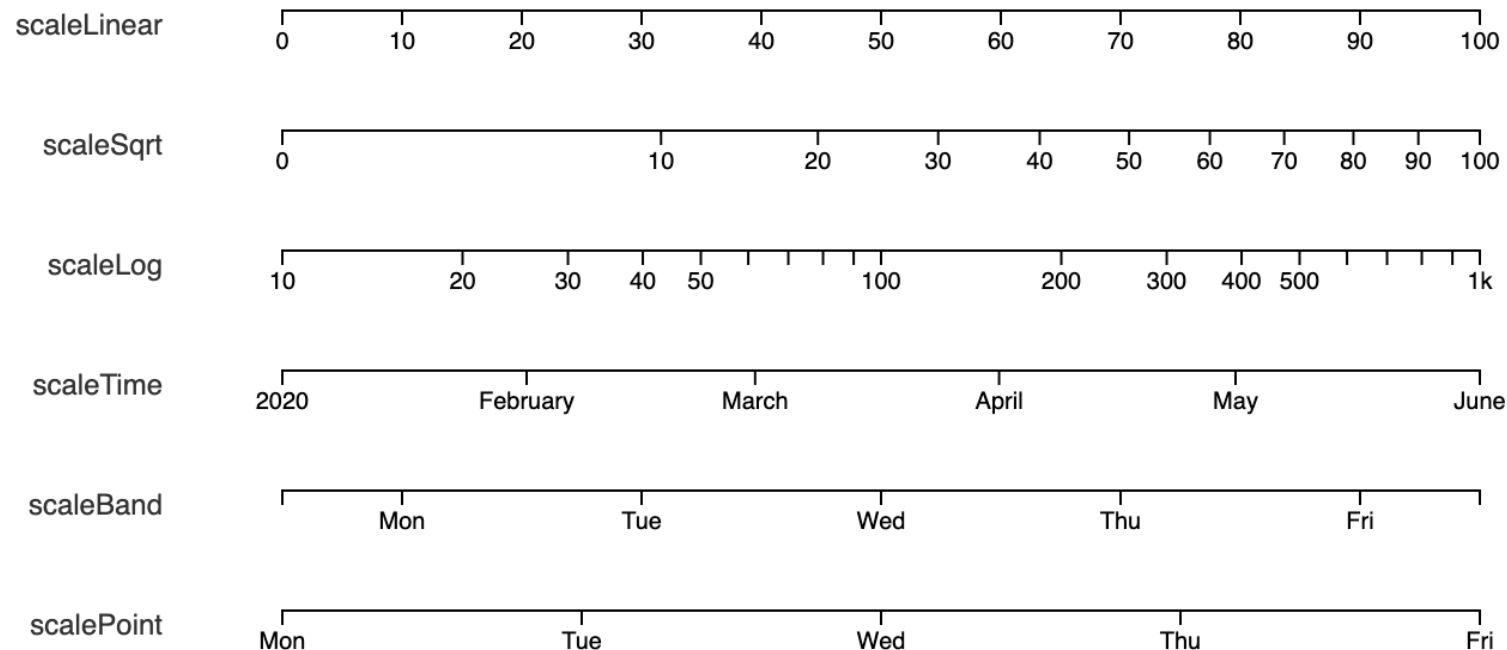
```
27  
28     <div id="fruit-chart">  
29     </div>  
30
```

```
23     let myData = [  
24         {name: "apple", price: 1.50},  
25         {name: "orange", price: 1.90},  
26         {name: "pear", price: 2.00}  
27     ]  
28  
29     let height = 300;  
30     let width = 600;  
31     let margin = 50;  
32  
33     let frame = d3.select("#fruit-chart")  
34         .append("svg")  
35         .attr("width", width)  
36         .attr("height", height);  
37  
38     let linearScale = d3.scaleLinear()  
39         .domain([0, 2.00])  
40         .range([height - margin, margin]);  
41  
42     frame.append("g")  
43         .attr("transform", `translate(${margin}, 0)`)  
44         .call(d3.axisLeft(linearScale));
```



D3 BASICS: AXES

- Many types of scale methods available



D3 BASICS: AXES

- `scaleBand` is particularly useful for bar charts
- `.bandwidth()` will return the width of each band in the scale
- `.paddingInner(percentage)` will set padding between bands

```
let bandScale = scaleBand()  
  .domain(['Mon', 'Tue', 'Wed', 'Thu', 'Fri'])  
  .range([0, 200]);
```

```
bandScale('Mon'); // returns 0  
bandScale('Tue'); // returns 40
```

```
bandScale.bandwidth(); // returns 40
```

```
bandScale.paddingInner(0.05);
```

```
bandScale.bandwidth(); // returns 38.38...  
bandScale('Mon');      // returns 0  
bandScale('Tue');      // returns 40.40...
```

D3 BASICS: AXES

- Your turn:
 - Add a horizontal axis to the fruit bars to identify which bar is which fruit

```
let myData = [  
  {name: "apple", price: 1.50},  
  {name: "orange", price: 1.90},  
  {name: "pear", price: 2.00}  
]
```

